

Molarium: When Molecular Design Software Becomes Generated Infrastructure

A perspective on adaptive scientific software, persistent molecular state, and discovery intelligence

Rafal P. Wiewiora

PsiThera

rafal.wiewiora@psithera.com

W. Patrick Walters

OpenADMET

pat.walters@omsf.io

Woody Sherman

PsiThera

woody.sherman@psithera.com

Perspective draft – August 2026

Abstract

Commercial computational drug-discovery software has traditionally bundled three sources of value: the interface, the scientific models, and the specialized expertise required to operate those models and translate their outputs into design decisions. Large language models, coding agents, and tool-using scientific agents are beginning to unbundle this paradigm. Interfaces, analyses, workflow logic, and substantial portions of scientific software can increasingly be generated around a particular scientist, project, and question, while agents can automate sophisticated workflows that previously required specialist users.

We argue that this does not commoditize computational drug discovery itself. It shifts the locus of differentiation. Faster execution does not improve an inaccurate model, extend an applicability domain, or make a weak prediction more useful. As the tool layer becomes cheaper and more fluid, competitive advantage should increasingly accrue to better and more generalizable models, proprietary experimental data, accumulated discovery intelligence, and the human judgment required to know what to ask and what to believe.

Molarium, available at <https://molarium.org>, provides a case study of this transition. Developed over an intensive weekend by a drug-discovery scientist working with a coding agent, this free open-source molecular workbench runs primarily in a standard web browser and combines local molecular visualization, editing, conformer search, molecular mechanics, molecular dynamics, machine-learned potentials, and protein-structure prediction. More importantly, Molarium preserves a versioned molecular state, user actions, method identity, provenance, and validation evidence beneath an interface that can change rapidly. This separation between fluid software and stable scientific state suggests a broader architecture for generated adaptive software in drug discovery.

Keywords: agentic software development; computational chemistry; molecular design; WebGPU; scientific validation; provenance; discovery intelligence

1 From fixed software products to generated adaptive software

Successful computational drug discovery depends on applying the right method at the right time, with appropriate settings, assumptions, and scientific context. Historically, this has been a tedious, error-prone, and often poorly reproducible process confined to a relatively small number of computational specialists. Docking requires careful protein preparation, protocol selection, appropriate constraints, and critical evaluation of the resulting poses before they can inform design. Molecular dynamics adds requirements for system preparation, force-field assignment, simulation protocols, compute infrastructure, trajectory analysis, and visualization, while free-energy calculations introduce further complexity in perturbation

design, sampling, convergence assessment, and alignment with experimental conditions. Similar layers of infrastructure surround cheminformatics, quantum chemistry, property prediction, molecular generation, and structural modeling.

Commercial scientific software has created considerable value by simplifying and integrating many of these activities. Graphical interfaces reduced technical barriers, workflow engines connected otherwise fragmented programs, and software packages encoded operational details needed to apply methods consistently. The same integration, however, coupled scientific questions to relatively fixed software environments. Project-specific improvements competed with broader product roadmaps, new analyses often required engineering support, and scientific users frequently adapted their questions to the capabilities and conventions of the available applications rather than the reverse.

Large language models and coding agents are beginning to change this relationship. A scientist with deep drug-discovery expertise but limited programming experience can increasingly describe an application, analysis, visualization, or workflow in natural language and have software constructed around the scientific need. Tool-using agents extend this capability by operating established methods, linking calculations, handling routine failures, and assembling results into forms that support decisions. Chemistry systems such as ChemCrow, ChemGraph, and related agentic frameworks demonstrate that language models can increasingly coordinate external tools and computational workflows rather than merely discuss them [1–4].

The consequence is not simply faster software development. It is a change in how drug hunters can interact with computational science. Rather than learning and adapting to a fixed software environment, scientists can increasingly shape the software around the question, the stage of the project, the data available at that moment, and their own preferred way of reasoning about the problem. A medicinal chemist optimizing a macrocycle need not use the same interface as a structural biologist interrogating a cryptic pocket or a program team deciding which compound to advance. The interface can become a temporary and adaptive view over a more durable scientific substrate.

Molarium emerged from this model. It began as a request for a better environment in which to inspect and manipulate molecular structures and expanded through use. Scientific objections became software changes: an implausible geometry triggered a change in molecular editing semantics; an unexpected energy led to an explicit numerical reference; ambiguity about a computational backend forced the implementation to expose which engine was actually running; and unsupported methods were converted into explicit limitations rather than hidden assumptions. The application evolved while the scientific question was still active.

2 Molarium and the value of a persistent molecular state

Molarium is an MIT-licensed, local-first molecular viewer, builder, and simulation workbench. Most supported calculations execute on the user’s device using WebAssembly or WebGPU, allowing molecular graphics, editing, cheminformatics, conformer exploration, molecular mechanics, molecular dynamics, machine-learned potentials, and selected protein-structure prediction workflows to coexist in a standard browser environment. The browser is important because it collapses installation and orchestration overhead and, for supported calculations, can avoid remote job submission, cloud compute charges, and application licensing simply to remain inside the design loop. WebAssembly provides a route for mature compiled scientific libraries to run locally, while WebGPU exposes modern accelerators through a portable browser API [5, 6].

The architectural choice that matters most, however, is not the browser itself. It is the preservation of a common scientific state. Molarium maintains explicit topology, coordinates, protonation state, chemical edits, preparation history, trajectories, model identity, and provenance as properties of the molecular session rather than allowing each view or calculation to silently construct its own interpretation of the molecule. The interface can therefore change rapidly without redefining the scientific object underneath it.

Software addition:
Visualize the “design trajectory” as a navigable history linking molecular states, user actions, calculations, evidence, and decision points.

Three task-specific views illustrate this separation. **View** emphasizes the molecular system, including protein, ligand, waters, ions, contacts, and selected residues. **Build** exposes chemical and geometrical interventions. **Run** emphasizes preparation findings, trajectories, and provenance. These views expose different aspects of the same evolving molecular state. The controls can change because the scientific questions change; topology, coordinates, method identity, and history do not change invisibly.



Figure 1: Molarium as an adaptive view over a persistent scientific state. Different views emphasize different design questions while the underlying molecular state, topology, coordinates, history, provenance, and computational context remain connected.

Editorial notes: Rename the three views **View**, **Design**, and **Simulate** in both this figure and the deployed application. Replace these panels with clearer screenshots that show what Molarium actually enables through an example design trajectory: make a three-dimensional chemical edit with live local minimization, launch a simulation, select and edit any resulting trajectory frame, and continue the design cycle. Consider adding a short section on the forthcoming core-constrained pose-preparation workflow—not conventional docking, but programmatic medicinal-chemistry transformations with Monte Carlo torsion optimization—as a medicinal-chemist-friendly route to preparing consistent poses for FEP.

This becomes particularly important when interfaces themselves are inexpensive to modify. If every generated view is free to redefine a molecule, select a different charge model, change protonation, alter preparation, or reinterpret a trajectory, flexibility becomes a source of scientific ambiguity. Generated adaptive software therefore demands a more explicit boundary between presentation and scientific state than fixed software typically provides.

Molarium takes this principle further by retaining the sequence of molecular states and user actions that produced the current session. Edits, selections, manipulations, calculations, and other interactions form an event history rather than leaving only the final molecular coordinates. The resulting architecture is closer to version control than to a traditional molecular project file. A molecular state can be associated with the actions that produced it, the calculation applied to it, and the evidence available at that point in the design process.

The analogy to “Git for molecules” is useful, although the scientific object is richer than source code. A drug-design trajectory contains not only structural changes but also changing hypotheses, model outputs, uncertainty, and experimental evidence. In a discovery setting, such a history could preserve which analogs were considered, which structures or properties influenced a design, which alternatives were rejected, and what was learned when a compound was eventually synthesized and tested. This creates a foundation for recording medicinal-chemistry design trajectories rather than merely storing the compounds that survived them. Much of the most valuable information in a discovery program lies in the path not taken, yet conventional databases usually preserve only endpoints.

The browser also provides a useful trust boundary for proprietary work. Molarium’s Local Lab mode can restrict network access and verify reviewed application assets, while supported calculations remain on the local device. Because privacy controls depend on deployment configuration, this architecture makes data movement explicit and auditable. Organizations can therefore evaluate and enforce how molecular data are handled rather than treating data transfer as an invisible property of a remote software stack.

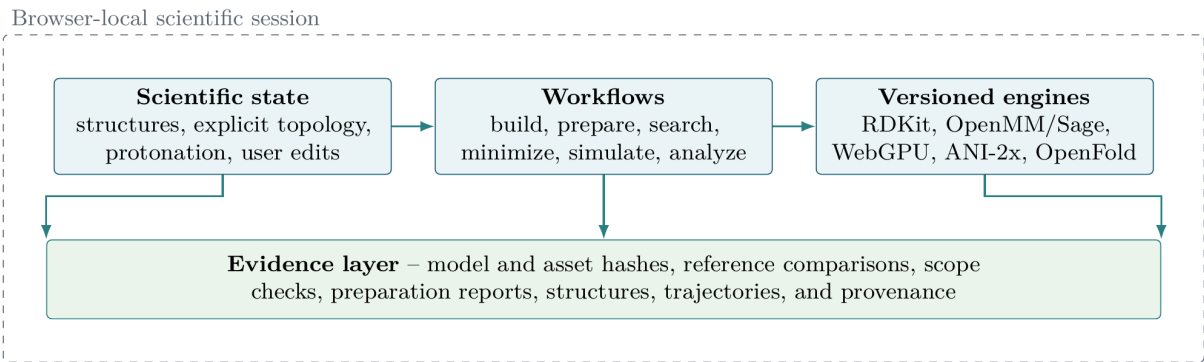


Figure 2: Molarium’s local-first architecture. Named computational engines act on an explicit molecular state while provenance and validation evidence remain attached. The same principle provides a foundation for versioned molecular histories in which scientific actions can be replayed and compared without allowing a new interface to redefine the underlying state.

3 Scientific contracts: provenance, validation, and falsifiability

Cheap, adaptive software creates a new scientific risk. An agentially generated workflow is most impactful and most liable precisely when it fills gaps fluently enough that the user no longer notices where the scientific contract ended. If a new analysis view can be created in minutes, that view must still expose which molecule, model, parameters, assumptions, and evidence produced the result. Otherwise, reducing the cost of software generation merely increases the rate at which scientific ambiguity can be created.

We therefore use the term **scientific contract** to describe the set of invariants and evidence that give a computational result meaning. A molecular mechanics result is not defined by the label “molecular mechanics.” It depends on the molecular state, parameterization, charge assignment, energy function, treatment of nonbonded interactions, constraints, boundary conditions, precision, integrator, and numerical implementation. A learned potential depends on its architecture, weights, supported chemistry, preprocessing, and training domain. A protein-structure prediction depends on the checkpoint, sequence and template information, inference protocol, and other model-specific choices. The interface may be fluid, but these scientific identities cannot be allowed to drift silently.

Molarium therefore keeps computational routes distinct even when they appear in the same workbench. RDKit provides chemical perception and conformer generation; OpenFF Sage defines a molecular-mechanics model; OpenMM Reference provides an independent numerical route for selected validation tests; STORMM-inspired WebGPU code advances homogeneous replicas; ANI-2x provides a separate machine-learned potential with a defined chemical domain; and OpenFold provides a separate protein-prediction model [7–12]. These methods can be compared, chained, or presented together, but the interface does not erase the fact that they make different approximations and answer different questions.

The same principle applies to workflow provenance. Scientific workflow research has long emphasized that reproducibility requires retaining the data, computational process, execution environment, and provenance that produced a result [15, 17]. More recent work formalizes workflow invariants and testable reproducibility criteria rather than treating provenance as passive metadata [16]. Generated adaptive software increases the importance of this idea because the workflow itself may no longer be a stable artifact defined months before execution. It can be created or modified during the scientific interaction.

This leads to a stronger requirement: generation of a scientific workflow should be accompanied by generation of the conditions under which its claims could be falsified. If an agent implements a new conformer-search strategy, the output should include not only the workflow but also the benchmark that would test whether it improves conformational coverage, the reference needed to evaluate it, the tolerance that defines agreement, and the cases that lie outside the intended scope. If an agent adds a numerical

kernel, it should generate the independent comparison and failure conditions required to establish that the implementation behaves as claimed.

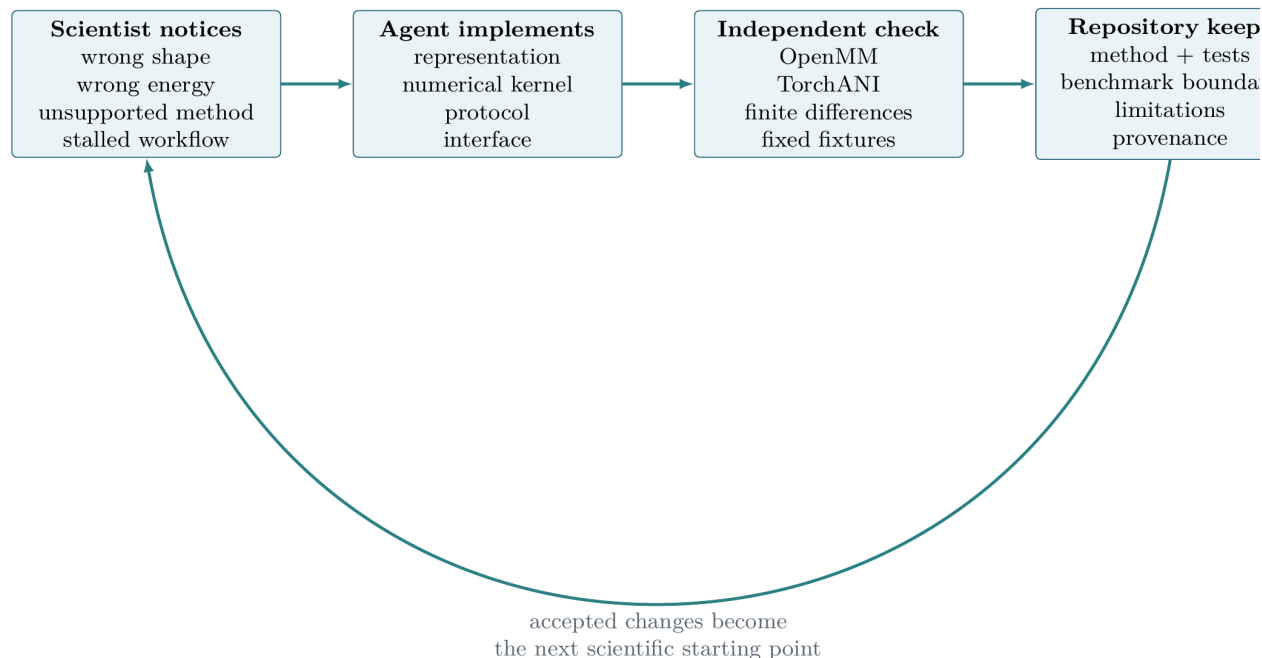


Figure 3: The scientist-agent build loop. Scientific needs and objections are translated into software changes, but each new capability is paired with explicit tests, reference evidence, limitations, and provenance. The ability to generate a workflow and the ability to specify how its claim could fail are treated as parts of the same scientific result.

Molarium’s development repeatedly followed this pattern. A chemically implausible geometry was not treated as a visual defect; it exposed a problem in representation and editing semantics. A questionable energy prompted comparison against an independent implementation. A discrepancy between CPU and GPU pathways revealed both a unit error and an incorrect assumption about which backend was executing. Requesting constrained dynamics led not only to implementation of SHAKE/RATTLE but also to tests against a reference pathway [13, 14]. In each case, the scientifically relevant output was not simply the additional feature. It was the feature together with the test that defined the boundary of the claim.

This distinction becomes essential as agentic software becomes more fluent. A unit error can generate a smooth trajectory. Two wrong implementations can agree if they share the same input error. A correct numerical implementation can instantiate an inappropriate physical model. Consequently, an attractive visualization or successful execution is the lowest rung of an evidence hierarchy, not evidence that a molecular prediction is correct.

When software is difficult to construct, implementation effort can obscure the distinction between building a method and validating it. When implementation becomes easy, the remaining scientific obligations become harder to ignore. The workflow, its provenance, and its falsification conditions should therefore be treated as one scientific object.

Clarify distinction: These should not sound like ordinary unit tests. Specify claim-defining scientific tests: independent references, quantitative tolerances, applicability limits, and explicit failure conditions. Consider “scientific contract tests” or “validation and falsification tests.”

4 Local computation changes the interaction model

The claim presented here is not that a web browser should replace production molecular simulation software or high-performance computing. Current native GPU codes remain substantially more capable for large

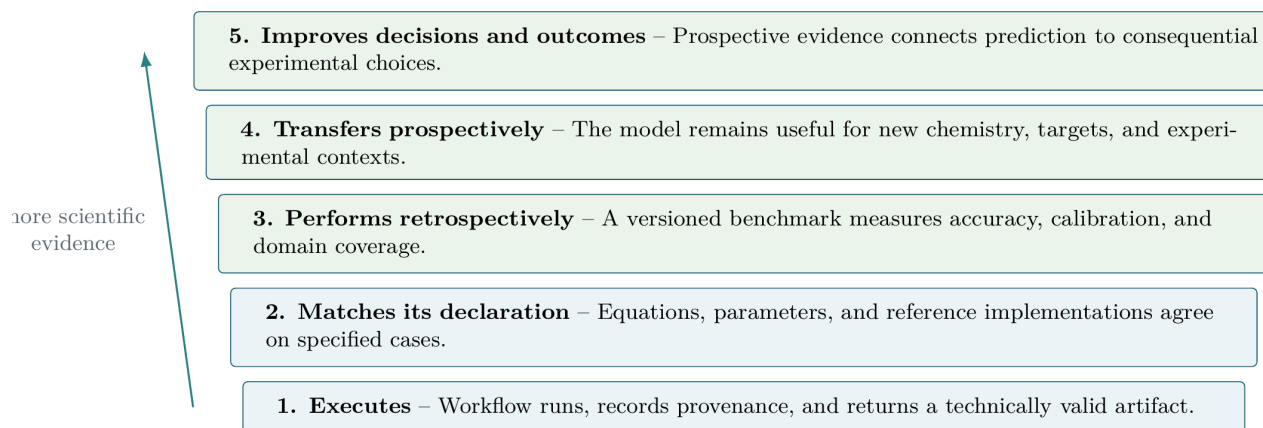


Figure 4: An evidence ladder for generated scientific software. Successful execution establishes less than numerical agreement; numerical agreement establishes less than retrospective predictive performance; retrospective performance establishes less than prospective transfer; and prospective accuracy is distinct from measurable improvement in drug-discovery decisions.

periodic systems, long trajectories, specialized force fields, advanced sampling, and many other demanding workflows. Molarium itself remains deliberately bounded, and several of its browser-native routes are experimental.

The more consequential observation is that a standard browser on commodity personal hardware can now host molecular calculations of sufficient sophistication to change the interaction model between scientist, software, and discovery team. Workflows that previously required separate installations, specialist environments, remote compute submission, substantial orchestration, or commercial applications can increasingly occur within the active design session. For supported workloads, the result, method identity, assumptions, validation evidence, and molecular history remain attached to the same scientific state rather than being fragmented across files, programs, notebooks, and job systems.

This has implications for both latency and economics. A scientist can interrogate a structure, modify a molecule, launch a local calculation, compare alternative models, and inspect the result without leaving the design environment. The marginal cost of many small calculations approaches the cost of the local hardware already present on the desk, rather than incurring a remote orchestration cycle or per-calculation cloud charge. An open-source workbench also removes application licensing as the gatekeeper to these supported workflows. These advantages are most important for calculations that sit directly in the design loop, where minutes of friction repeated hundreds of times can matter more than the peak performance of an isolated benchmark.

Local execution also changes the distribution of control. The scientist can determine which analysis should exist, how data should be juxtaposed, and which calculation should be available at the point of decision. Molarium itself is evidence of this change: a scientist did not need to wait for every useful workflow to appear on a conventional product roadmap. Missing interfaces and bounded implementations could be added while the underlying question was still active.

The important caveat is that lower latency increases the need for scientific discipline. Cheap execution can tempt users to run more calculations simply because they are available. The goal is not maximal computational throughput; it is maximal scientific leverage. The useful question is whether a calculation changes a design decision, reduces meaningful uncertainty, or identifies an experiment that would discriminate among competing hypotheses.

5 What becomes scarce when the tool layer becomes cheap

Molarium helps separate three layers of value that have historically been packaged together. The first is the **tool layer**: graphical interfaces, visualization, format conversion, wrappers, workflow logic, reports, job control, data plumbing, and bounded numerical implementation. These capabilities remain important and require maintenance, but their marginal reproduction cost is falling quickly as coding agents improve. A scientist can increasingly generate a project-specific view or connect an existing computational method without waiting for the corresponding feature to become part of a commercial release.

The second is the **model layer**: force fields, quantum mechanical methods, docking and scoring functions, free-energy models, machine-learned potentials, protein-structure predictors, molecular representations, property models, parameterization, weights, uncertainty calibration, and the experimental evidence that defines their useful domain. This layer does not become inexpensive simply because invocation becomes easier. A docking model with an inadequate scoring function remains inadequate when invoked by an agent. A force field with a systematic error becomes more dangerous, not less, if an automated workflow generates thousands of confident predictions from it. A machine-learned property model does not acquire extrapolative validity because a language model describes its answer fluently. A protein-structure predictor does not become reliable for a particular conformational state because it is available through a better interface.

As the mechanics of invoking a model become less scarce, differences among models become more important. Easier access does not improve predictive quality, and obtaining poor answers faster may save human time while doing little to increase the probability of success. This distinction is particularly important in drug discovery, where the objective is frequently to extrapolate into new chemistry, new structural states, and new biological contexts rather than interpolate within a well-sampled training distribution [18–20].

The third layer is **discovery intelligence**: proprietary experimental data, negative results, project history, assay context, structural interpretation, design rationale, internal protocols, expert judgment, program objectives, and the remembered reasons that apparently attractive ideas failed. Molarium intentionally contains little of this third layer because it is a public workbench built primarily from public scientific components. That absence clarifies the opportunity. An organization could build generated adaptive interfaces over the same architectural principles while connecting them to internal structures, assay data, pharmacokinetics, target biology, medicinal-chemistry reasoning, and program objectives.

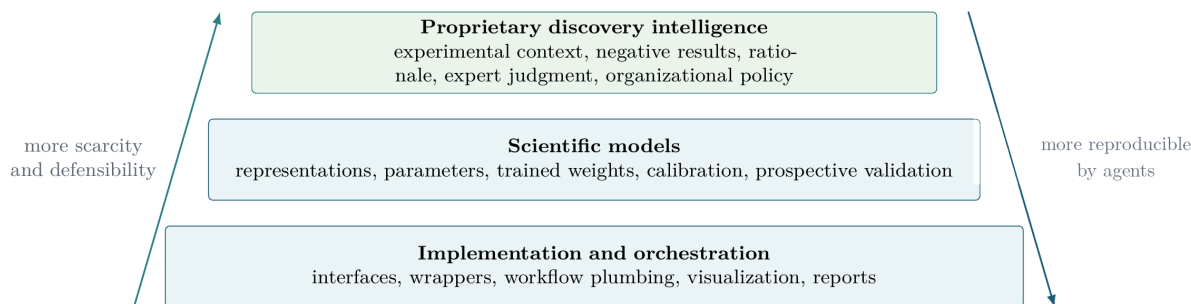


Figure 5: Three interdependent layers of value in generated adaptive molecular design. Implementation and orchestration are becoming increasingly reproducible by coding agents. Scientific models remain differentiated by accuracy, representation, domain coverage, calibration, and validation. Proprietary discovery intelligence combines experimental data with the context and decision history that make those data useful.

The versioned molecular history described above creates an important bridge into this third layer. Traditional compound databases are very good at recording what was made and measured. They are much worse at recording what was considered, why one design was preferred over another, which model or structure influenced that decision, and what evidence was visible at the time. A persistent action history can begin

to preserve this decision trajectory. Over repeated design cycles, the organization can accumulate not only data but the relationship between hypotheses, interventions, predictions, decisions, and outcomes.

This is more than provenance in the narrow sense of reproducing a calculation. It is **decision provenance**. A future system should be able to answer not only “How was this free-energy value computed?” but also “Why was this molecule proposed?”, “Which alternatives were rejected?”, “What evidence changed the team’s view?”, and “Which prediction ultimately proved useful or misleading?” That information is difficult to reconstruct retrospectively and highly specific to an organization’s science. It may therefore become one of the most durable forms of discovery intelligence.

Agentic systems provide a mechanism to operationalize this knowledge. A protein-preparation skill can encode how an organization treats protonation, crystallographic waters, alternate conformations, unusual residues, and escalation of ambiguous cases. A free-energy skill can encode accepted transformations, convergence criteria, diagnostics, and failure modes. A compound-design skill can combine measured and predicted properties with uncertainty, synthetic feasibility, novelty, and current program objectives. The important asset is not the prose of the prompt; it is the versioned decision process, including its inputs, models, assumptions, controls, domain restrictions, escalation conditions, and accountable scientific owner.

6 The changing role of scientific expertise

There is an apparent paradox in this transition. As software becomes easier to construct and operate, human judgment becomes more important. Traditional computational expertise includes substantial operational knowledge: how to prepare a particular input, which menu exposes a setting, how to translate files between programs, how to recover a failed job, or how to reproduce a standard analysis. These skills remain useful, but an increasing fraction can be encoded in software and agents. The broader automation literature has long recognized that reducing routine operation can increase the importance of human expertise precisely when the system encounters ambiguous or abnormal conditions [21].

The highest-value expertise therefore shifts toward selecting the right question, deciding the appropriate level of model fidelity, defining the acceptable tradeoff among speed, throughput, and accuracy, recognizing implausible results, choosing meaningful controls, and determining which result would actually influence a project decision. This is a different emphasis from learning how to operate conventional scientific software. The scientist closest to the underlying discovery problem—whether a medicinal chemist, structural biologist, mechanistic biologist, pharmacologist, or computational scientist—can increasingly interact directly with the relevant data and calculations rather than requiring every question to pass through a software intermediary.

We do not view this as the disappearance of computational scientists. The more plausible outcome is a shift in leverage. Less specialist time needs to be spent constructing repetitive interfaces, transferring files, or manually operating routine workflows. More can be spent on model development, molecular physics, parameterization, sampling, uncertainty, validation, and difficult edge cases where the established workflow fails. Once that expertise is made explicit, part of it can be encoded into reusable scientific skills that broaden access without pretending that the underlying science has become simple.

Molarium’s development illustrates the division of labor. The coding agent could inspect a repository, refactor software, implement numerical kernels, add tests, and repeat the loop rapidly. The scientist had to decide which result looked chemically impossible, whether a reference was genuinely independent, whether a discrepancy mattered, and what evidence would change belief. The agent compressed the distance between recognizing a scientific need and changing the software; it did not remove the need to recognize the scientific need.

The bottleneck therefore shifts from operation toward epistemology. What approximation is acceptable? What is the appropriate tradeoff between cost and fidelity? Which result deserves an independent control? Is the reference actually independent? Is a model being used outside its applicability domain? What

uncertainty should remain visible? What experiment would distinguish competing hypotheses? The ability to implement an answer quickly increases the value of asking the right question.

7 Limitations and outlook

Molarium is a case study rather than a controlled experiment in software productivity or drug-discovery performance. Its development involved a small number of scientists, one evolving codebase, and a particular coding-agent workflow. There was no randomized comparison with a conventional software-development team, and the project cannot establish a universal productivity multiplier or determine what fraction of commercial scientific software can be regenerated reliably.

Its scientific scope is also deliberately bounded. Current browser-native molecular mechanics does not reproduce the full capability of mature native simulation environments. General periodic simulation, particle-mesh Ewald electrostatics, barostats, membranes, unrestricted protein parameterization, arbitrary plugins, and other advanced capabilities remain outside the current browser-native scope. ANI-2x retains its declared elemental and molecular domain, and browser OpenFold support is narrower than general production protein-prediction workflows. Hardware and browser differences also remain relevant to numerical behavior and performance. The public repository explicitly labels experimental routes and records validation boundaries rather than presenting the workbench as a replacement for production simulation software.

Most importantly, implementation agreement is not molecular prediction. A browser kernel can reproduce a declared force-field calculation while the force field remains inaccurate for the chemistry under study. Two implementations can agree while sharing the same problematic parameterization. A conformer workflow can explore more structures without improving recovery of experimentally relevant states. A successful retrospective benchmark can fail prospectively. These are not peripheral qualifications; they are central to the thesis.

The future suggested by Molarium is therefore not one in which every scientist maintains a private molecular-modeling package or every commercial application is replaced by generated code. It is a future in which the interface between scientist and computation becomes more fluid while the scientific state, models, assumptions, evidence, and decision history become more explicit. Project teams can construct temporary views around specific questions; internal models can coexist with open-source methods and commercial services; and the user experience no longer needs to be owned by the provider of each underlying calculation.

The durable infrastructure in such an environment is not the screen. It is the versioned molecular state, the computational model and its parameters, the applicability domain, the validation and falsification tests, the provenance of the result, and the accumulated discovery intelligence governing how that result should be interpreted. Generated software makes the presentation layer easier to change. It also makes these scientific invariants more important to preserve.

8 Conclusion

Molarium began as a request for a better environment in which to inspect and manipulate molecules. It developed rapidly into a local-first molecular-design workbench incorporating topology-aware editing, protein preparation, conformer exploration, molecular mechanics, molecular dynamics, learned potentials, protein-structure prediction, browser-native GPU computation, provenance, and reference validation. The speed of that development is technically interesting, but the broader lesson is the change in architecture and in the location of scientific value.

Software can increasingly be generated around the scientific question and the user’s specific preferences. That makes the interface cheap and fluid, but it makes the quality of the underlying models, validation,

Community invitation:

Introduce the GitHub repository as a place to collaborate on a living regeneration benchmark: select capabilities described in papers or public specifications, attempt independent reconstruction, test against references, and record what can and cannot be reproduced reliably.

Clarify relevance:

Of course Molarium does not enlarge ANI-2x’s domain. The point should be that generated wrappers and easier access must preserve, expose, and enforce the underlying model’s applicability limits rather than implying broader scientific validity.

Molarium interface addition:

Add a persistent scientific-contract panel showing the active molecular state, model and parameter versions, applicability limits, validation status, assumptions, and provenance regardless of which generated view is open.

and decision history more important, not less. The scarce assets shift from operating software toward trustworthy models, proprietary discovery intelligence, and the human judgment required to know what to ask and what to believe.

Molarium also suggests that molecular design environments should preserve more than the latest structure. A versioned record of molecular states, user actions, calculations, assumptions, and results can capture the trajectory by which scientific decisions were made. Coupled to explicit scientific contracts and falsification tests, that history can make generated adaptive software not only easier to build, but more reproducible, extensible, and scientifically accountable.

When software is difficult to construct, implementation effort can hide the distinction between building a method and validating it. When implementation becomes easy, the remaining scientific obligations become much harder to ignore. The opportunity is therefore not simply to generate more software. It is to put adaptable computational capability directly in the hands of the scientists who understand the problem, while preserving the models, evidence, and decision history that determine whether the answer should be trusted.

The tool layer is becoming cheap and adaptive. Scientific advantage moves toward better models, better discovery intelligence, and better decisions.

AI assistance and availability

GPT-5.6 Sol was used extensively as a coding agent during the development of Molarium under human scientific direction, inspection, testing, and revision. Language-model assistance was also used in restructuring and drafting this manuscript. The authors remain responsible for the scientific claims, validation, source attribution, and final text.

Molarium is available at <https://molarium.org>. Its source code and technical documentation are publicly available at <https://github.com/rafwiewiora/molarium>.

A submitted version relying on quantitative benchmark claims should be accompanied by a frozen validation ledger recording the relevant commit, fixtures, model and asset hashes, browser and hardware environment, numerical precision, and timing boundaries.

References

- [1] Bran, A. M.; Cox, S.; Schilter, O.; Baldassari, C.; White, A. D.; Schwaller, P. Augmenting Large Language Models with Chemistry Tools. *Nature Machine Intelligence* **2024**, *6*, 525–535.
- [2] Boiko, D. A.; MacKnight, R.; Kline, B.; Gomes, G. Autonomous Chemical Research with Large Language Models. *Nature* **2023**, *624*, 570–578.
- [3] Pham, T. D.; Tanikanti, A.; Keçeli, M. ChemGraph as an Agentic Framework for Computational Chemistry Workflows. *Communications Chemistry* **2026**, *9*, 33.
- [4] MacKnight, R.; Boiko, D. A.; Regio, J. E.; Gallegos, L. C.; Neukomm, T. A.; et al. Rethinking Chemical Research in the Age of Large Language Models. *Nature Computational Science* **2025**, *5*, 715–726.
- [5] Haas, A.; Rossberg, A.; Schuff, D. L.; et al. Bringing the Web up to Speed with WebAssembly. *Proc. PLDI* **2017**, 185–200.
- [6] W3C GPU for the Web Working Group. *WebGPU Specification*, 2025.
- [7] Riniker, S.; Landrum, G. A. Better Informed Distance Geometry: Using What We Know to Improve Conformation Generation. *J. Chem. Inf. Model.* **2015**, *55*, 2562–2574.

- [8] Boothroyd, S.; Madin, O. C.; Mobley, D. L.; et al. Development and Benchmarking of Open Force Field 2.0.0: The Sage Small-Molecule Force Field. *J. Chem. Theory Comput.* **2023**, *19*, 3251–3275.
- [9] Eastman, P.; Galvelis, R.; Peláez, R. P.; et al. OpenMM 8: Molecular Dynamics Simulation with Machine Learning Potentials. *J. Phys. Chem. B* **2024**, *128*, 109–116.
- [10] Cerutti, D. S.; Wiewiora, R. P.; Boothroyd, S.; Sherman, W. STORMM: Structure and Topology Replica Molecular Mechanics for Chemical Simulations. *J. Chem. Phys.* **2024**, *161*, 032501.
- [11] Devereux, C.; Smith, J. S.; Huddleston, K. K.; et al. Extending the Applicability of the ANI Deep Learning Molecular Potential to Sulfur and Halogens. *J. Chem. Theory Comput.* **2020**, *16*, 4192–4202.
- [12] Ahdrizt, G.; Bouatta, N.; Floristean, C.; et al. OpenFold: Retraining AlphaFold2 Yields New Insights into Its Learning Mechanisms and Capacity for Generalization. *Nature Methods* **2024**, *21*, 1514–1524.
- [13] Ryckaert, J.-P.; Ciccotti, G.; Berendsen, H. J. C. Numerical Integration of the Cartesian Equations of Motion of a System with Constraints: Molecular Dynamics of n-Alkanes. *J. Comput. Phys.* **1977**, *23*, 327–341.
- [14] Andersen, H. C. RATTLE: A “Velocity” Version of the SHAKE Algorithm for Molecular Dynamics Calculations. *J. Comput. Phys.* **1983**, *52*, 24–34.
- [15] Cohen-Boulakia, S.; Belhajjame, K.; Collin, O.; et al. Scientific Workflows for Computational Reproducibility in the Life Sciences: Status, Challenges and Opportunities. *Future Gener. Comput. Syst.* **2017**, *75*, 284–298.
- [16] Pritchard, N. J.; Wicenec, A. Formal Definition and Implementation of Reproducibility Tenets for Computational Workflows. *Future Gener. Comput. Syst.* **2025**, *166*, 107684.
- [17] Wilkinson, M. D.; Dumontier, M.; Aalbersberg, I. J.; et al. The FAIR Guiding Principles for Scientific Data Management and Stewardship. *Sci. Data* **2016**, *3*, 160018.
- [18] Bender, A.; Cortés-Ciriano, I. Artificial Intelligence in Drug Discovery: What Is Realistic, What Are Illusions? Part 1. *Drug Discov. Today* **2021**, *26*, 511–524.
- [19] Bender, A.; Cortés-Ciriano, I. Artificial Intelligence in Drug Discovery: What Is Realistic, What Are Illusions? Part 2. *Drug Discov. Today* **2021**, *26*, 1040–1052.
- [20] Wigh, D. S.; Goodman, J. M.; Lapkin, A. A. A Review of Molecular Representation in the Age of Machine Learning. *WIREs Comput. Mol. Sci.* **2022**, *12*, e1603.
- [21] Bainbridge, L. Ironies of Automation. *Automatica* **1983**, *19*, 775–779.